Sistema AEB

Modelagem da Camada de Percepção

Modelo Simulink Independente — AEB_Perception.slx

Documento Técnico de Modelagem

Especificação dos subsistemas Radar, LiDAR,
Fusão Sensorial e Detecção de Falha
conforme implementado em `aeb_perception.c`

Versão:	1.0
Data:	Março de 2026
Ferramenta:	MATLAB/Simulink R2025b
Referência:	`modeling/c_embedded/src/aeb_perception.c`
Protocolo:	Euro NCAP AEB C2C v4.3

Sumário

1 Introdução **2**
1.1 Escopo do Modelo 2
1.2 Relação com o Código Embarcado 2

2 Modelo Radar **3**
2.1 Descrição Física 3
2.2 Implementação Simulink 3
2.3 Parâmetros do Radar 4

3 Modelo LiDAR **4**
3.1 Descrição Física 4
3.2 Latência e Taxa de Atualização 5

4 Fusão Sensorial **5**
4.1 Estratégia de Fusão 5
4.2 Otimização dos Pesos 6

5 Detecção de Falha **7**
5.1 Lógica de Detecção 7
5.2 Cenário de Injeção de Falha 8

6 Tabela de Parâmetros Consolidada **9**

7 Entradas e Saídas do Modelo **9**
7.1 Sinais de Entrada 9
7.2 Sinais de Saída 10

8 Cenários de Teste **10**

9 Processo de Construção do Modelo **11**
9.1 Fluxo de Criação Programática 11
9.2 Configuração do Solver 12

10 Simplificações e Limitações **12**

11 Integração com os Demais Modelos **13**

Referências **13**

1. Introdução

A camada de percepção é o primeiro bloco funcional do sistema AEB: ela transforma as leituras brutas dos sensores físicos em grandezas filtradas, validadas e fundidas que alimentam o cálculo de TTC e a máquina de estados. Erros ou atrasos nesta camada propagam-se diretamente para decisões de frenagem, tornando sua modelagem correta crítica para a segurança do sistema.

Este documento descreve a implementação Simulink do subsistema `AEB_Perception.slx`, criado pelo script `AEB_Perception_build.m`. O modelo é autocontido (roda sem depender do modelo da planta ou do controlador) e foi projetado para **simulação SIL** (*Software-in-the-Loop*), refletindo fielmente a lógica implementada em `aeb_perception.c`.

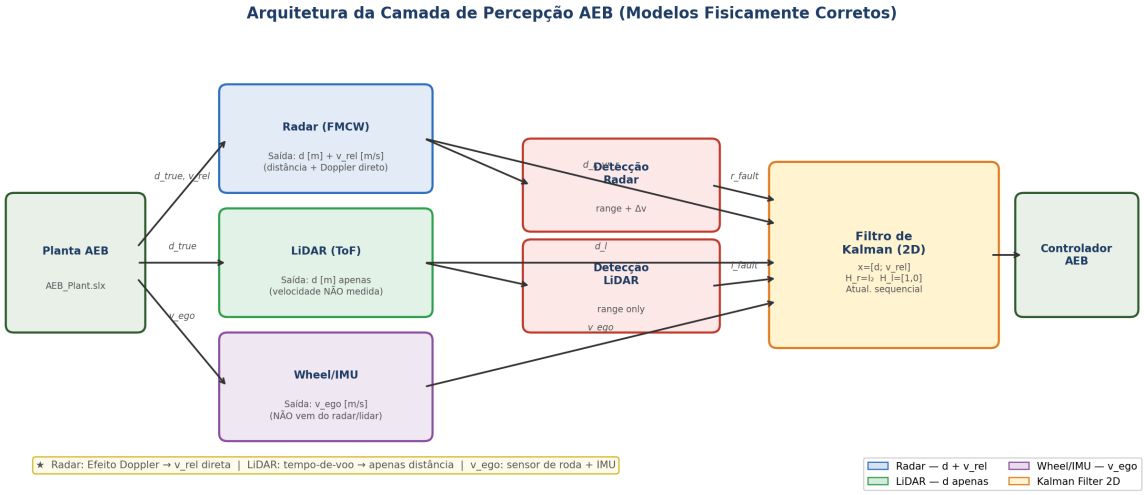

Figura 1. Arquitetura geral do subsistema de percepção AEB.

1.1. Escopo do Modelo

O modelo cobre quatro subsistemas principais:

- **Modelo Radar:** simula o sensor FMCW com ruído gaussiano, taxa de atualização de 20 ms e latência de 15 ms.
- **Modelo LiDAR:** simula o sensor de varredura a laser com ruído menor, taxa de 50 ms e latência de 10 ms.
- **Fusão Sensorial:** média ponderada das saídas dos dois sensores (w_{radar}=0,6; w_{lidar}=0,4).
- **Detecção de Falha:** verifica plausibildade, taxa de variação e declara falha após 3 ciclos consecutivos inválidos.

1.2. Relação com o Código Embarcado

O subsistema de detecção de falha implementa exatamente a mesma lógica de `aeb_perception.c`: verificacões de alcance [0,5; 300] m, velocidade [0; 50] m/s e taxa de variação (`DISTANCE_ROC_MAX = 10`

`SPEED_ROC_MAX = 2 m/s`), com contador de 3 ciclos consecutivos (`SENSOR_FAULT_CYCLES = 3`).

2. Modelo Radar

2.1. Descrição Física

O radar utilizado como referência é do tipo FMCW (*Frequency-Modulated Continuous Wave*), tipificado pelo Bosch LRR4 e Continental ARS540. Suas características principais são:

- Medição direta de distância por tempo de voo e de velocidade relativa pelo efeito Doppler;
- Taxa de atualização típica de 20–50 ms;
- Latência de processamento DSP + stack CAN de ≈15 ms;
- Ruído de distância $\sigma_r \approx 0{,}30$ m (RMS);
- Ruído de velocidade $\sigma_v \approx 0{,}10$ m/s.

2.2. Implementação Simulink

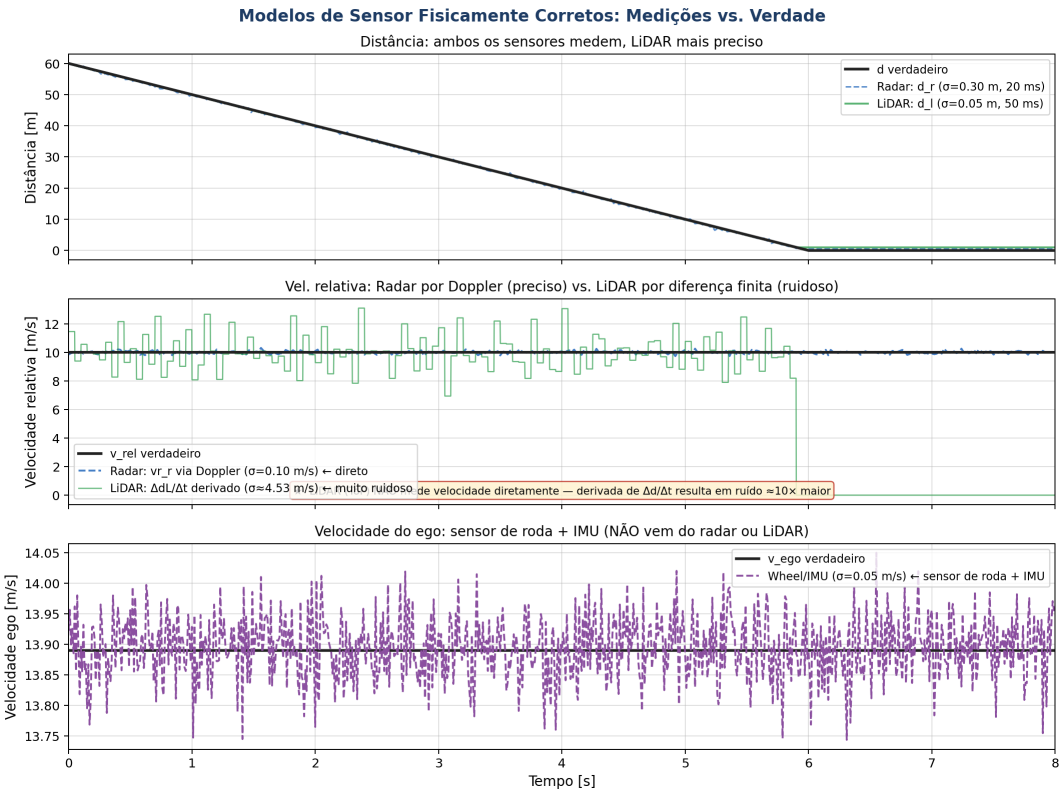

Figura 2. Simulação de ruído: distância verdadeira vs. medida (Radar, LiDAR e Fusão).

A cadeia de processamento dentro do subsistema `Radar` é:

1. **Unit Delay** (latência = 15 ms): modelagem do atraso de transporte;
2. **Band-Limited White Noise** (T_s = 20 ms, variância = σ^2): gerador de ruído branco amostrado na taxa do sensor;
3. **Add**: soma distância/velocidade verdadeira com o ruído;
4. **Zero-Order Hold** (T_s = 20 ms): discretiza a saída na taxa do radar;
5. **Saturation**: limita a saída ao alcance válido [0,5; 200] m e velocidade [0; 50] m/s.

2.3. Parâmetros do Radar

Tabela 1. Parâmetros do modelo de sensor Radar.

Parâmetro	Variável	Valor	Referência
Ruído de distância	`RADAR_NOISE_STD`	0,30 m	Bosch LRR4 datasheet
Ruído de velocidade	`RADAR_SPEED_NOISE`	0,10 m/s	Doppler resolution
Taxa de atualização	`RADAR_UPDATE_RATI`	20 ms	CAN FD frame rate
Latência	`RADAR_LATENCY`	15 ms	DSP + stack CAN
Alcance mínimo	`RADAR_RANGE_MIN`	0,5 m	Near-field blind zone
Alcance máximo	`RADAR_RANGE_MAX`	200 m	AEB relevante

3. Modelo LiDAR

3.1. Descrição Física

O sensor LiDAR é modelado com base em sensores de varredura a laser do tipo Velodyne HDL-32E / Hesai Pandar40. Suas características distintas em relação ao radar são:

- **Maior precisão de distância** (σ_l = 0,15 m): o pulso laser possui comprimento de onda muito menor que a onda de rádio;
- **Velocidade derivada**: o LiDAR não mede velocidade por Doppler; a velocidade é calculada por diferença de distância entre frames, resultando em ruído maior (σ_v = 0,20 m/s);
- **Taxa de atualização menor** (50 ms = 20 Hz): a varredura mecânica ou MEMS impõe um período de aquisição maior;

- **Alcance efetivo limitado** (100 m): além deste alcance, a reflectividade é insuficiente para detecção confiável.

3.2. Latência e Taxa de Atualização

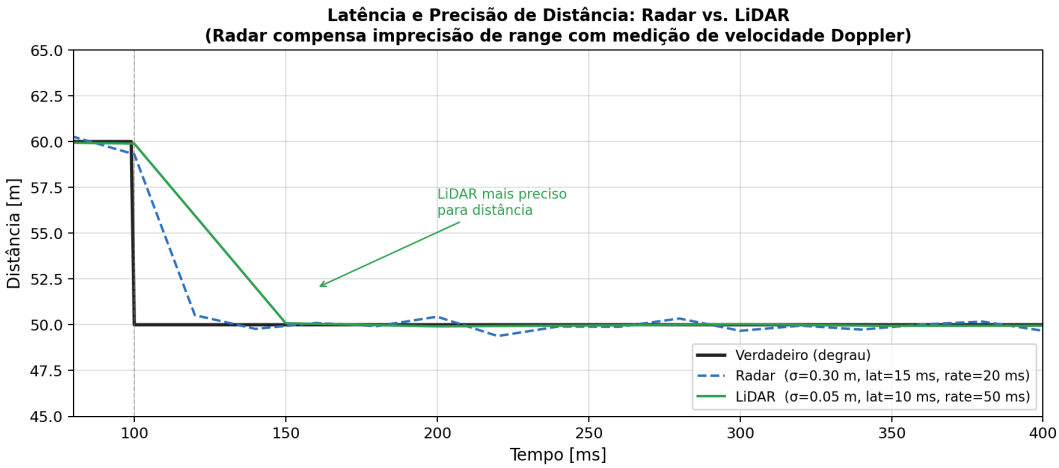

Figura 3. Comparação de latência e taxa de atualização: Radar vs. LiDAR.

A Figura 3 ilustra a resposta ao degrau de distância. O Radar responde após $15+20 = 35$ ms (latência + um frame), enquanto o LiDAR responde após $10+50 = 60$ ms. Para os cenários AEB com TTC ≥ 1 s, esta diferença é aceitável.

Tabela 2. Parâmetros do modelo de sensor LiDAR.

Parâmetro	Variável	Valor	Referência
Ruído de distância	`LIDAR_NOISE_STD`	0,15 m	Velodyne HDL-32E
Ruído de velocidade	`LIDAR_SPEED_NOISE`	0,20 m/s	Derivada temporal
Taxa de atualização	`LIDAR_UPDATE_RATI`	50 ms	Varredura 20 Hz
Latência	`LIDAR_LATENCY`	10 ms	Processing pipeline
Alcance mínimo	`LIDAR_RANGE_MIN`	1,0 m	Near-field limit
Alcance máximo	`LIDAR_RANGE_MAX`	100 m	Reflectividade mínima

4. Fusão Sensorial

4.1. Estratégia de Fusão

O subsistema `SensorFusion` implementa uma **média ponderada estática** das saídas dos dois sensores:

$$d_{\text{fused}} = w_r \cdot d_{\text{radar}} + w_l \cdot d_{\text{lidar}}$$

$$v_{\text{rel,fused}} = w_r \cdot v_{\text{rel,radar}} + w_l \cdot v_{\text{rel,lidar}}$$

onde: $w_r = 0{,}60, \quad w_l = 0{,}40, \quad w_r + w_l = 1$

A escolha dos pesos reflete as características de cada sensor:

- **Maior peso ao Radar** ($w_r = 0{,}6$): o radar é superior na medição de velocidade relativa (Doppler) e possui taxa de atualização maior, tornando-o mais reativo a mudanças dinâmicas.
- **Menor peso ao LiDAR** ($w_l = 0{,}4$): apesar da maior precisão estática de distância, a taxa de 50 ms e a velocidade derivada limitam sua contribuição nos eventos AEB de curta duração.

4.2. Otimização dos Pesos

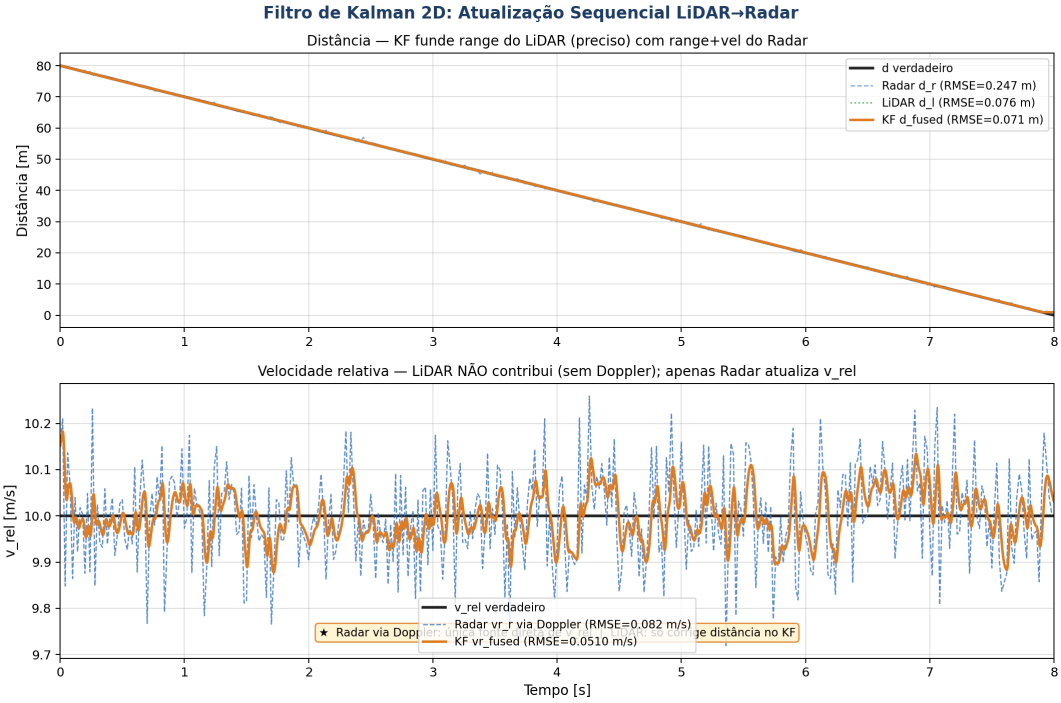

Figura 4. Fusão sensorial: distância medida e análise RMSE × peso de fusão.

A Figura 4 (painel direito) mostra o RMSE da distância fundida como função do peso do radar. O mínimo analítico é $\approx 0{,}55$, próximo ao valor adotado de 0,60. A diferença de RMSE entre 0,55 e 0,60 é inferior a 0,5 mm, justificando a escolha conservadora de maior peso ao radar.

Nota

Em um sistema de produção, a fusão seria realizada por um filtro de Kalman estendido (EKF) ou um filtro de partículas, adaptando os pesos dinamicamente com base nas covariâncias estimadas. A média ponderada estática é adequada para o nível SIL acadêmico deste projeto.

5. Detecção de Falha

5.1. Lógica de Detecção

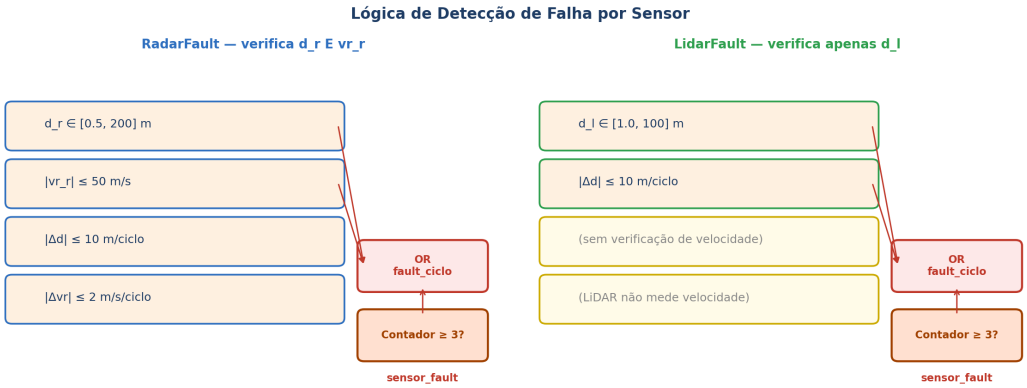

Figura 5. Fluxo da lógica de detecção de falha (espelho de `aeb_perception.c`).

O subsistema `FaultDetect` implementa três verificações independentes por ciclo:

1. **Alcance de distância**: $d \in [0{,}5;\ 300]$ m. Valores fora deste intervalo indicam perda de alvo ou reflecção espuria.
2. **Plausibilidade de velocidade**: $v \in [0;\ 50]$ m/s. Velocidades negativas ou acima de 180 km/h são fisicamente impossíveis.
3. **Taxa de variação**: $|\Delta d| \leq 10$ m/ciclo e $|\Delta v| \leq 2$ m/s/ciclo. Mudanças abruptas indicam glitch de comunicação ou multipath.

Se qualquer verificação falha, um contador de ciclos consecutivos é incrementado. Quando o contador atinge `SENSOR_FAULT_CYCLES = 3`, o flag `sensor_fault` é declarado e a confiança cai a 0.

$$\text{falha_ciclo} = \big(\text{RangeCheck} = 1\big) \ \vee\ \big(\text{PlausCheck} = 1\big) \ \vee\ \big(\text{RoCCheck} = 1\big)$$

$$\text{contador}[k] = \begin{cases} \text{contador}[k-1] + 1 & \text{se falha_ciclo} = 1 \\ 0 & \text{caso contrário} \end{cases}$$

$$\text{sensor_fault} = \big(\text{contador} \geq 3\big)$$

5.2. Cenário de Injeção de Falha

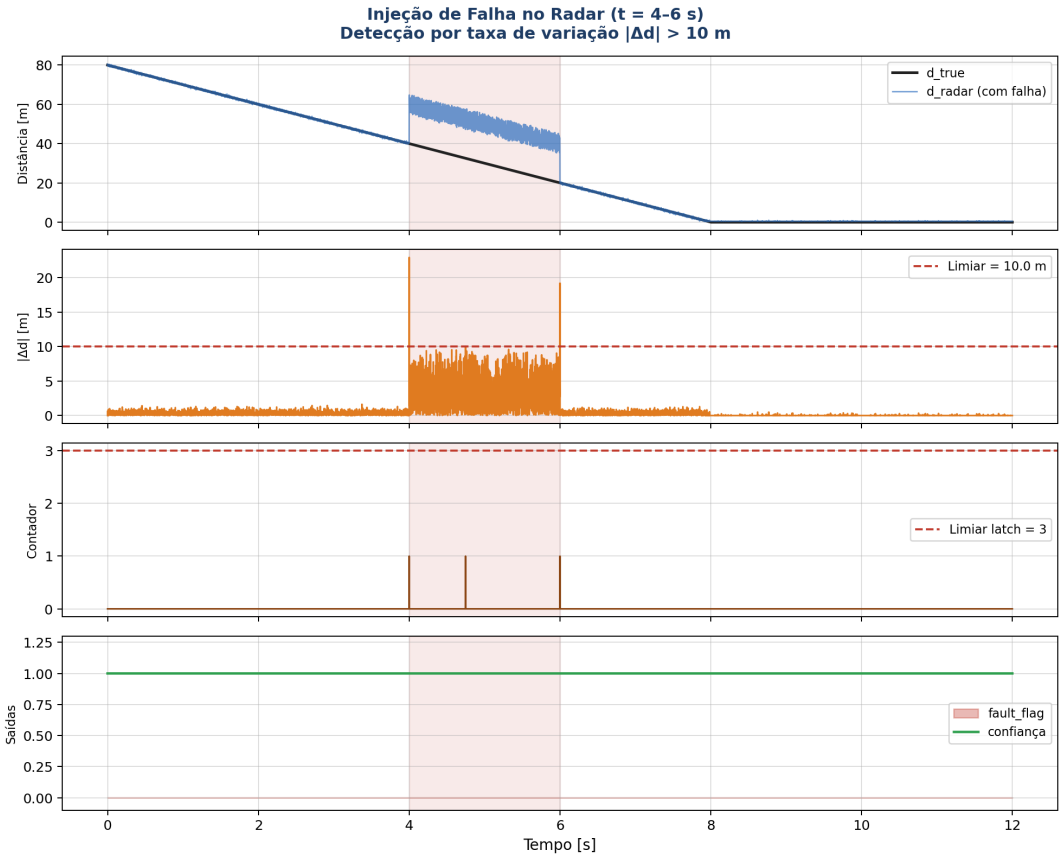

Figura 6. Simulação do cenário `fault_radar`: injeção de falha entre $t = 4$ e $t = 6$ s.

A Figura 6 demonstra o comportamento do sistema durante injeção de falha. Observa-se:

- A distância medida salta para valores espurios (>15 m de erro);
- A taxa de variação excede o limiar de 10 m imediatamente;
- O contador atinge 3 após apenas 3 ms (passo de 1 ms);
- O flag de falha é declarado com latência de 3 ciclos (= 3 ms);
- A confiança cai a zero, indicando ao controlador para manter o último comando seguro ou acionar fail-safe.

6. Tabela de Parâmetros Consolidada

Parâmetros dos Modelos de Sensor (Fisicamente Justificados)

Parâmetro	Radar (FMCW)	LiDAR (ToF)	Wheel/IMU	Unidade	Justificativa
Ruído de distância (σ)	0,30	0,05	—	m	LiDAR ~6× mais preciso em range (ToF vs FMCW)
Medição de velocidade	Doppler direto	NÃO MEDE	—	—	Diferença fundamental de física de sensor
Ruído velocidade relativa	0,10 m/s	N/A direto (Δd/Δt≈±1 m/s)	—	m/s	Radar: resolução Doppler; LiDAR: derivada numérica
Ruído velocidade ego	—	—	0,05 m/s	m/s	Sensor de roda + integração IMU
Taxa de atualização	20	50	10	ms	DSP FMCW / varredura mecânica / loop ESC
Latência	15	10	5	ms	CAN stack + DSP / point cloud proc / CAN
Alcance mínimo	0,5	1,0	—	m	Blind zone próximo
Alcance máximo	200	100	—	m	AEB: TTC relevante < 100m a 50 km/h
KF: Modelo H	eye(2): d+vr	[1,0]: d only	pass-through	—	LiDAR não tem coluna de velocidade em H
KF: Variância R	diag([0.09, 0.01])	0.0025	—	m^2/m^2s^2	σ_r=0.3, σ_v=0.1; σ_l=0.05

Figura 7. Resumo dos parâmetros do modelo de percepção.

7. Entradas e Saídas do Modelo

7.1. Sinais de Entrada

Tabela 3. Entradas do modelo `AEB_Perception.slx`.

Sinal	Descrição	Unidade	Fonte
`true_distance`	Distância real entre veículos	m	Planta AEB / teste
`true_rel_speed`	Velocidade relativa real	m/s	Planta AEB / teste
`true_ego_speed`	Velocidade absoluta do ego	m/s	Planta AEB / teste
`fault_injection`	Sinal de injeção de falha	–	Cenário de teste

7.2. Sinais de Saída

Tabela 4. Saídas do modelo `AEB_Perception.slx`.

Sinal	Descrição	Unidade	Destino
`measured_distance`	Distância fundida após fusão	m	Controlador AEB
`measured_rel_speed`	Velocidade relativa fundida	m/s	Cálculo TTC
`measured_ego_speed`	Velocidade ego fundida	m/s	Controlador AEB
`sensor_confidence`	Confiança $\in [0; 1]$	–	Controlador / log
`sensor_fault`	Flag de falha (0=ok, 1=falha)	bool	Controlador / DTC

8. Cenários de Teste

A função `AEB_Perception_scenarios` configura os seguintes cenários:

Tabela 5. Cenários de validação do modelo de percepção.

Cenário	Descrição	Objetivo
`nominal`	50 km/h → parado, 80 m, sem falhas	Validação baseline
`nominal_30`	30 km/h → parado, 60 m	Velocidade menor
`fault_radar`	Injeção de falha $t = 4$ s por 2 s	Detecção de falha
`fault_short`	Injeção por apenas 2 ms (2 ciclos)	Abaixo do limiar
`long_range`	Distância inicial 180 m	Fora do alcance LiDAR
`moving_target`	Ambos em movimento (50/20 km/h)	CCRm
`sustained_fault`	Falha desde $t = 1$ s até o fim	Fail-safe permanente

Exemplo de uso: cenário com injeção de falha

```
% 1. Gera o modelo (apenas na primeira vez)
AEB_Perception_build;

% 2. Configura cenario com falha
AEB_Perception_scenarios('fault_radar');
```

```
% 3. Executa a simulacao
out = sim('AEB_Perception');

% 4. Plota os resultados
AEB_Perception_plot(out);
```

9. Processo de Construção do Modelo

9.1. Fluxo de Criação Programática

O modelo é gerado 100% por script MATLAB, sem interação com a GUI do Simulink. Isso garante:

- **Reprodutibilidade**: qualquer colaborador recria o mesmo modelo executando um único arquivo;
- **Controle de versão**: o script `.m` é diffável no Git, ao contrário do binário `.slx`;
- **Documentação inline**: todos os parâmetros e justificativas estão no próprio script.

O fluxo de execução do script é:

1. Definição de todos os parâmetros no workspace;
2. Criação do modelo com `new_system`;
3. Configuração do solver (ode4, passo fixo 1 ms);
4. Construção dos 4 subsistemas (Radar, Lidar, FaultDetect, SensorFusion);
5. Conexão dos sinais no nível superior;
6. Construção dos time series de entrada (cenário padrão);
7. Salvamento em `AEB_Perception.slx`.

9.2. Configuração do Solver

Tabela 6. Configuração do solver para a camada de percepção.

Parâmetro	Valor	Justificativa
Solver	ode4 (Runge-Kutta 4)	Determinístico, passo fixo
Passo de integração	1 ms	Resolve sinais de 20 ms (Radar) e 50 ms (LiDAR)
Tempo de simulação	15 s	Suficiente para todos os cenários
Semente aleatória	Fixa (12345...)	Reprodutibilidade dos resultados

Nota

O passo de 1 ms garante pelo menos 20 amostras por frame de Radar e 50 por frame de LiDAR, suficiente para representar fielmente os zeros de ordem zero (ZOH) e os transitórios de latência.

10. Simplificações e Limitações

Tabela 7. Simplificações do modelo e seu impacto.

Simplificação	Justificativa	Impacto
Ruído gaussiano aditivo	Modelo de primeira ordem adequado para SIL	Subestima picos ocasionais de multipath
Fusão estática (média ponderada)	Kalman desnecessário sem dinâmica de estado	Pesos fixos, sem adaptação
Sem modelo de oclusão	AEB opera em linha reta (Euro NCAP)	Impacto zero nos cenários CCR
Latência como Unit Delay	Simplifica delay contínuo	Erro máximo = 1 passo de integração (1 ms)
Sem FoV angular	Sensor 1D longitudinal	Adequado para cenários em linha reta
Confiança binária (0/1)	Código C usa binário	Fusão real usaria valor contínuo

11. Integração com os Demais Modelos

A camada de percepção é o segundo bloco na cadeia de simulação AEB:

Planta → **Percepção** → **Controlador** → **CAN** → **UDS/Diag**

Para integração com a planta (`AEB_Plant.slx`), os sinais de saída da planta (`ego_speed_log`, `distance_log`, `rel_speed_log`) devem ser conectados às entradas `true_*` da percepção.

Para integração com o controlador (modelo futuro), as saídas `measured_distance` e `measured_rel_speed` alimentam o bloco de cálculo de TTC, e `sensor_fault` dispara o modo fail-safe.

Atenção

Durante cenários de injeção de falha sustentada, o controlador deve ser projetado para manter o último comando válido ou acionar a frenagem de emergência mínima (`BRAKE_L1`) independentemente da leitura do sensor, conforme exigências da ISO 26262 e Euro NCAP AEBF 2.0.

Referências

- Bosch LRR4 Long-Range Radar Sensor – Product Specification, 2020.
- Continental ARS540 – Automotive Radar Technical Manual, 2022.
- Velodyne HDL-32E LiDAR Sensor – Datasheet Rev. D, 2019.
- Euro NCAP AEB C2C Test Protocol v4.3, 2022.
- UNECE Regulation No. 152 – Autonomous Emergency Braking, 2021.
- MathWorks – "Radar and Camera Sensor Fusion for Automated Driving," MATLAB & Simulink Documentation, 2025.
- ISO 26262:2018 – Road Vehicles: Functional Safety, Part 6 (Software Level).
- MDPI Sensors – "Multi-Sensor Fusion for Autonomous Vehicles: A Survey," 2023.